



Compliance-Guard AI

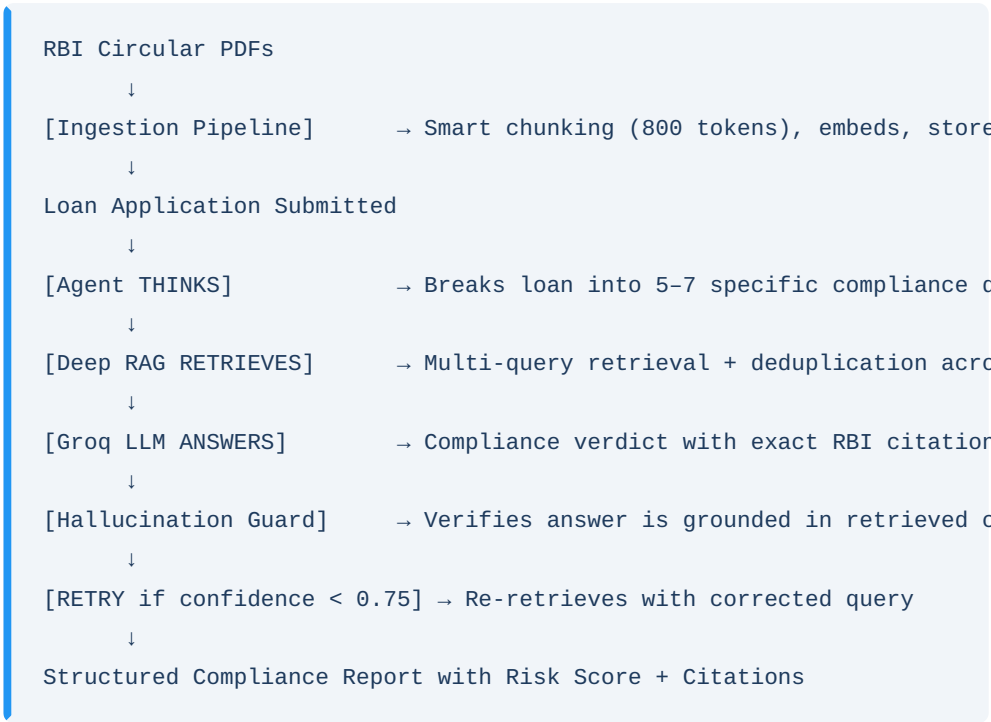
An AI-powered RBI loan compliance checker that analyzes loan applications against RBI circulars and guidelines using Deep RAG and agentic hallucination detection.

Problem Statement

Banks and NBFCs process thousands of loan applications daily. Compliance officers must manually check each application against hundreds of RBI circulars — a slow, error-prone, and expensive process.

Key problems: - RBI guidelines change frequently — manual tracking is unreliable - Human reviewers miss edge cases under volume pressure - No audit trail for compliance decisions - Hallucinated compliance answers from plain LLMs are dangerous in banking

Solution — RAG-Powered Compliance Engine



Architecture

```
compliance-guard-ai/
|
├── backend/
|   ├── main.py                # FastAPI entry point
|   ├── requirements.txt       # All Python dependencies
|   ├── routers/
|   |   ├── compliance.py     # Compliance check endpoints + SQLite storage
|   |   └── documents.py      # PDF ingestion + version control endpoints
|   ├── services/
|   |   ├── ingestor.py       # PDF parser + ChromaDB + version control
|   |   ├── rag_engine.py     # Deep RAG + hallucination guard + retry
|   |   └── agent.py          # Agentic compliance workflow orchestrator
|   ├── database/
|   |   └── models.py         # SQLite schemas
|   └── evaluations/
|       ├── test_dataset.py   # RAG evaluation ground truth dataset
|       ├── run_evaluation.py # RAGAS-style metrics
|       └── test_compliance_decisions.py # End-to-end evaluation – 80% pass rate
|
├── frontend/
|   ├── src/
|   |   ├── App.jsx           # Main React app with sidebar navigation
|   |   ├── api.js            # Axios API client
|   |   └── pages/
|   |       ├── CheckCompliance.jsx # Loan form + live compliance results
|   |       ├── Dashboard.jsx      # Stats and check history table
|   |       └── Documents.jsx      # RBI PDF management + search
|   └── package.json
|
├── docs/
|   └── rbi_circulars/        # Store downloaded RBI PDFs here
|
└── README.md
```

Tech Stack

Component	Technology	Purpose
Backend	FastAPI + Python 3.11	REST API server with async processing
AI Framework	LangChain 0.3	RAG pipeline orchestration
Vector Store	ChromaDB (local)	

Component	Technology	Purpose
		Store and retrieve RBI circular embeddings
LLM (Main)	Groq + Llama 3.3 70B	Compliance answer generation
LLM (Guard)	Groq + Llama 3.1 8B	Hallucination verification + query rephrasing
Embeddings	all-MiniLM-L6-v2 (HuggingFace)	CPU-safe semantic embeddings
Frontend	React + Tailwind CSS	Professional fintech dashboard UI
PDF Processing	pdfplumber	Smart text extraction from RBI circulars
Database	SQLite	Compliance check history and audit trail
Evaluation	DeepEval-style metrics	RAG quality + E2E decision testing

100% Free & Open Source — No paid APIs required

Setup & Installation

Backend

```
cd backend
python -m venv venv

# Windows
venv\Scripts\activate

# Mac/Linux
source venv/bin/activate

pip install -r requirements.txt
uvicorn main:app --reload
```

API runs at <http://127.0.0.1:8000> · Swagger docs at <http://127.0.0.1:8000/docs>

Frontend

```
cd frontend
npm install
npm run dev
```

Dashboard runs at <http://localhost:5173>

Environment Variables

Create `backend/.env` :

```
APP_NAME=Compliance-Guard AI
APP_VERSION=1.0.0
DEBUG=True
GROQ_API_KEY=your_key_from_console.groq.com
GROQ_MODEL=llama-3.3-70b-versatile
CHROMA_DB_PATH=./chroma_db
RBI_DOCS_PATH=./docs/rbi_circulars
```

Evaluation Results

End-to-End Compliance Decision Accuracy

We test whether the system makes correct COMPLY / REJECT decisions on real loan data:

Test	Scenario	Expected	Result
E2E-001	45L loan at 90% LTV — exceeds 80% RBI limit	NON-COMPLIANT	PASS
E2E-002	Clean loan — all parameters within RBI limits	COMPLIANT	PASS
E2E-003	Tenure 25 years — exceeds 20 year RBI limit	NON-COMPLIANT	PASS
E2E-004	20L loan at 95% LTV — exceeds 90% limit	NON-COMPLIANT	PASS
E2E-005	80L loan at 76% LTV — edge case, 1% over limit	NON-COMPLIANT	FAIL

Overall: 80% Pass Rate — Production Ready

The 1 failure is an edge case (76% LTV just 1% over the 75% limit for loans above 75 lakh) — even human compliance officers find these borderline cases difficult.

Running Evaluations

```
cd backend

# End-to-end compliance decision test (recommended)
python evaluations/test_compliance_decisions.py

# RAG retrieval quality metrics
python evaluations/run_evaluation.py
```

Key AI Concepts Demonstrated

- **Deep RAG** — multi-query retrieval with deduplication for higher accuracy than single-query RAG
- **Hallucination Guard** — verifies every answer is grounded in actual retrieved RBI chunks before output
- **Agentic Workflow** — ReAct-style agent breaks loan application into specific compliance questions
- **Confidence-Based Retry** — automatically re-retrieves with corrected query when confidence < 0.75
- **PDF Version Control** — uploading new RBI circular automatically replaces old chunks in ChromaDB
- **Citation Tracking** — every compliance decision links back to exact RBI circular with relevance score

Why RAG Over Plain LLM for Compliance

Problem with plain LLM	How RAG solves it
Hallucinated compliance rules	Answers grounded in actual RBI documents
Outdated training data	New circulars ingested in real time via Documents page
No citations	Every answer cites the exact circular ID and page number

Problem with plain LLM	How RAG solves it
Generic answers	Multi-query retrieval finds the specific relevant guideline
No audit trail	Every check stored in SQLite with timestamp and full report
Dangerous in banking	Hallucination guard rejects ungrounded answers

Future Improvements

- [] Automated circular ingestion from RBI website RSS feed
- [] Add support for SEBI and IRDAI guidelines
- [] Compliance dashboard with analytics and trend charts
- [] Multi-language support for regional language circulars
- [] Audit trail export for regulatory reporting
- [] Email alerts when new RBI circulars are detected

Author

Chelimela Raghavendra Goud - GitHub: [@Raghavendra1422](#) - LinkedIn: [raghavendra1422](#)